Programmatic Access to University Compute Resources

Using UBELIX and GPUStack in a software project

David Herrmann | Sebastian Flick

Data Science Lab, University of Bern

Data Talk

The project

- `plant3d`: film a plant → 3D model in the browser
- Reconstruction is Gaussian splatting (requires a GPU)
- Eleven metadata fields per recording, typed in a greenhouse, on a phone
- We want to record what people say while they film to extract metadata

All of this should ideally be done in-house.

Live demo

Three problems

1. Speech to text
   the words out of the recording
2. Text to metadata
   eleven fields out of the words
3. Programmatic access to UBELIX
   the cluster has no API

1

Speech to text

Try decoding locally, fail over to the server

- The model needs the audio. The video is up to 500 MB.
- Web Audio API: decode, downmix, resample → a few hundred kB
- Some browsers do not provide that API
- On failure, `ffmpeg` extracts the audio on the server

```
 try {
  transcriptionFile = await extractAudioFromVideo(file);
} catch {
  usedVideoFallback = true;  // let the server do it
}
```

GPUStack existed

- Initially, the only feasible option seemed to be a subscription (contract, research data leaving the university).
- Then we learned about `gpustack.unibe.ch`
- University hardware, free for members, OpenAI-compatible
- `faster-whisper-large-v3` for the transcript. One URL, one token.

The university's GPUStack provides various reasonably up-to-date GenAI models.

2

Text to metadata

From transcript to form

transcript → `gpt-oss-120b`, same GPUStack → eleven fields

```
"Maize, cultivar Ronaldinio,
three weeks, still vegetative,
greenhouse, two litre pots,
standard soil.  Operator Anna."
```

```
 species:  Zea mays
cultivar:  Ronaldinio
plant_age:  3 weeks
pot_volume:  2
operator:  Anna
genotype:  null
```

Prompt does the work: only the JSON object, unknown fields null, `temperature 0`. Form stays editable.

Demo

The prompt

```
 System:
You translate natural language descriptions of plant experiments
into a JSON object with the following format:
{ "type": "object", "properties": {
    "title":      {"type": "string", "description": "A short descriptive title"},
    "species":    {"type": "string", "description": "Plant species name"},
    "cultivar":   {"type": "string", "description": "Plant cultivar or variety"},
    "plant_age":  {"type": "string", "description": "Age of the plant"},
    "pot_volume": {"type": "number", "description": "Volume of the pot in liters"},
    ... genotype, growth stage, environment, substrate, treatments, operator
  }, "additionalProperties": false }
Rules:
- ONLY output the json object, do not include any explanations or extra text.
- If information is missing or uncertain, make the field null.
```

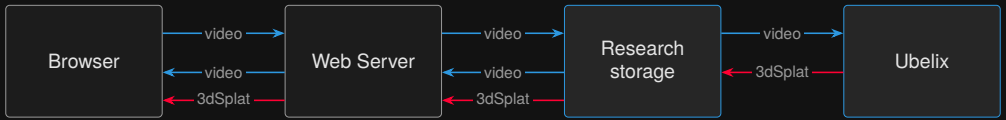
The schema is trimmed here, the rest is the prompt as it ships. `temperature 0`, and `<think>` blocks get stripped before we parse.

3

Programmatic access to UBELIX

The video data flow

- Video data
  1. has to be accessible from within UBELIX
  2. should stay available after it has been processed
  3. should not live on the web server
- We therefore use the university's **Research Storage (RS)** to store the videos — the RS is accessible from UBELIX



UBELIX has no side door

- How people use it today: SSH in, write a batch script, type `sbatch`, come back later
- A workflow for a person at a terminal
- No API, no tokens, no webhooks

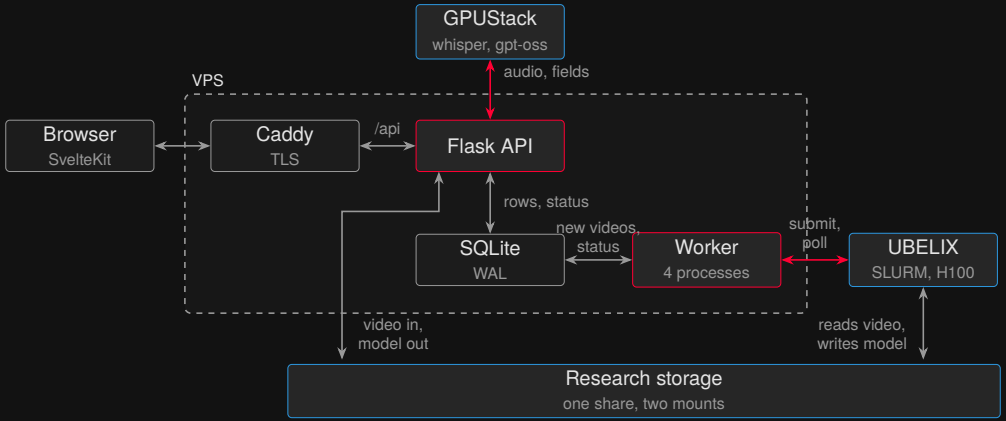
Nobody we asked had launched UBELIX jobs from an application.

So we use the front door

- Service account, own ed25519 key
- `paramiko`: SSH library for Python
- `sbatch -parsable` produces (parseable) JSON output → read job id
- `sacct -json` again JSON output → extract current processing status
- Four concurrent workers submit jobs over SSH and poll their ID

PENDING **PROCESSING** **PROCESSED** **FAILED**

The whole path



Four containers on one host. Only the proxy has a port open.

The script for the UBELIX job

- The script on UBELIX that turns a video into a 3D splat is written and maintained by the project team
- Our responsability is to place the video data where the script expects it
- When the script changes, we (ideally) don’t have to change anything

Our takeaways

1. Check what the university already runs. In our case: LLMs on GPUStack, Speech-to-Text models and Embedding models.
2. Even though UBELIX doesn't have a web API, service account plus `-json` gets you far.

Thank you

plant3d.ips.unibe.ch

David Herrmann | Sebastian Flick